

AI Specialization 1: Learnability Models, Bandits, and Reinforcement Learning

AI Specialization 1: SMU

Learnability models: PAC and online. Learnability of conjunctions and disjunctions. Multi-armed bandit problem. Reinforcement learning.

Sources used: official CTU CourseWare page for Symbolic Machine Learning, which lists reinforcement learning and computational learning theory as course parts; CTU state-exam topic list; standard learning theory and RL material.

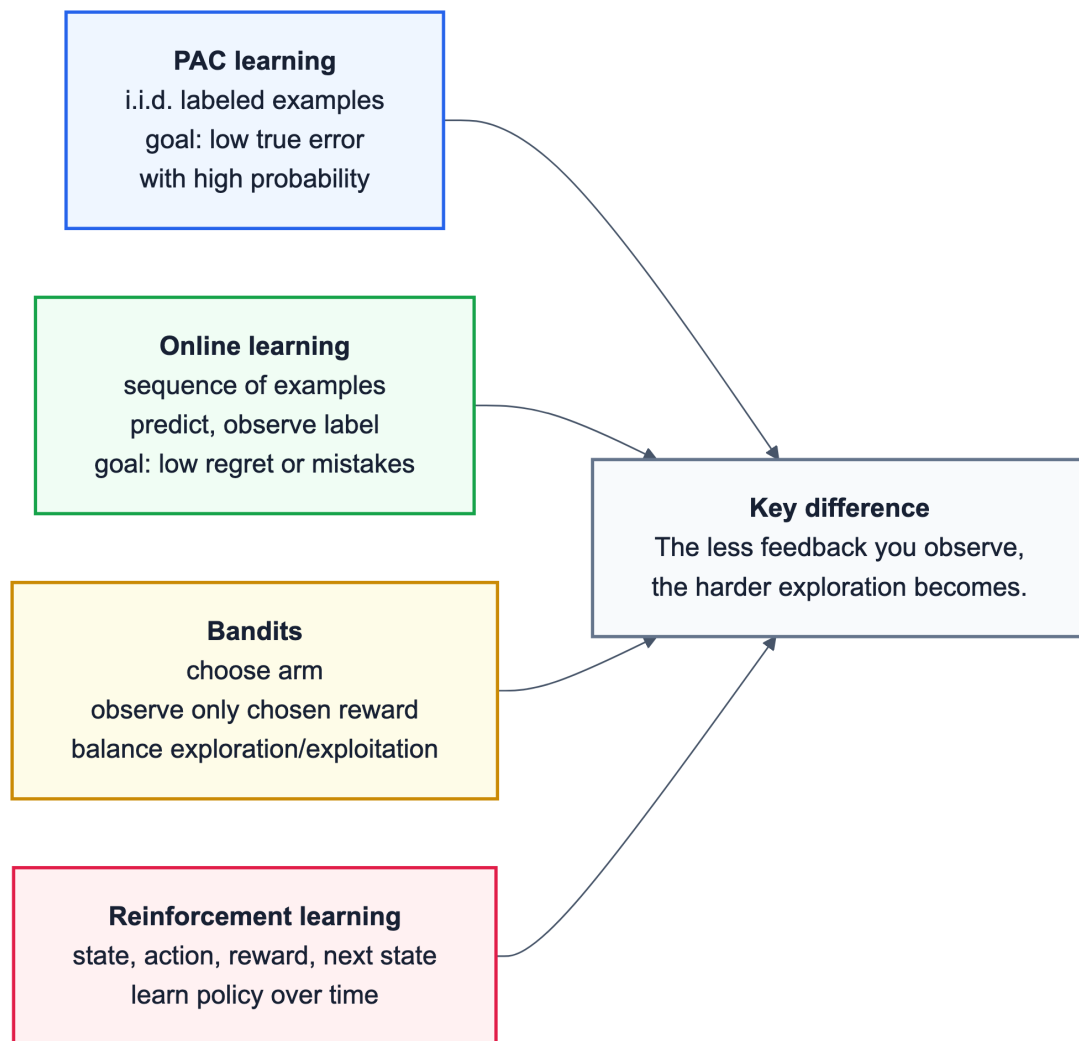


Figure 1: Learning-feedback settings: PAC, online learning, bandits, and reinforcement learning differ mainly in what feedback is observed.

SMU in Plain English

SMU is about learning from feedback. The main question is always: what feedback do we see, what is the learner trying to minimize, and how much exploration is needed?

1. Identify feedback.

Full label, mistake, chosen reward, or delayed reward?

2. Name the goal.

Low error, few mistakes, low regret, high return.

3. Give the update.

Eliminate, explore, estimate values, or improve policy.

PAC learning

Batch examples from a fixed distribution.

- Goal: low true error with high probability.
- Needs sample complexity in epsilon and delta.
- VC dimension controls learnability.

Online learning

Predict one example at a time.

- Goal: few mistakes or low regret.
- Can be adversarial sequence.
- Conjunction/disjunction learners eliminate inconsistent literals.

Bandits

You only observe reward for the arm you pulled.

- Explore/exploit tension is central.
- Regret compares against best fixed arm.
- UCB = average reward plus uncertainty bonus.

Reinforcement learning

Actions change future states and rewards.

- Value iteration assumes known model.
- TD/Q-learning learn from sampled transitions.
- Policy search optimizes policy directly.

Key contrast

Less feedback makes learning harder.

- PAC: label for sampled example.
- Bandit: only chosen reward.
- RL: chosen reward plus next state, delayed consequences.

How to answer

Always say the feedback model first.

- What is observed?
- What is optimized?
- What algorithm handles exploration?

Exam sentence: In bandits the learner does not see rewards of unchosen arms, so algorithms like UCB add an uncertainty bonus to try promising but under-sampled actions.

Three traps

- PAC assumes i.i.d. data; online learning may be adversarial.
- Bandit regret is not the same as supervised classification error.
- Q-learning estimates action values, not just state values.

Use this page to keep the feedback settings separate.

1. PAC and Online Learnability

PAC model

PAC means **P**robably **A**pproximately **C**orrect. We have:

- instance space X ,
- labels $Y = \{0, 1\}$, or $\{-1, +1\}$,
- unknown distribution D over X ,
- unknown target concept $c \in C$,
- training examples $(x_i, c(x_i))$ drawn i.i.d. from D .

A learner outputs hypothesis $h \in H$. Its true error is

$$err_D(h) = \Pr_{x \sim D}[h(x) \neq c(x)].$$

A concept class C is PAC-learnable by H if for every $\epsilon, \delta \in (0, 1)$ and every distribution D , after

$$m = poly(1/\epsilon, 1/\delta, n, size(c))$$

i.i.d. examples, the learner outputs h such that

$$\Pr[err_D(h) \leq \epsilon] \geq 1 - \delta.$$

It is **efficiently PAC-learnable** if the learner also runs in polynomial time.

Online model

In online learning, examples arrive sequentially. At round t :

1. learner receives x_t ,
2. predicts $\hat{y}_t$,
3. true y_t is revealed,
4. learner updates its state.

No i.i.d. assumption is needed; the sequence may be adversarial. Performance is measured by mistakes or regret.

Mistake-bound model: for realizable sequences labeled by some $c \in C$, learner should make at most $M(n)$ mistakes.

Regret model: compare cumulative loss to best fixed hypothesis in hindsight:

$$R_T = \sum_{t=1}^T \ell(\hat{y}_t, y_t) - \min_{h \in H} \sum_{t=1}^T \ell(h(x_t), y_t).$$

Sublinear regret $R_T = o(T)$ means average performance converges to the best comparator.

PAC vs. online

Main differences:

Aspect	PAC	Online
Data	i.i.d. samples from fixed distribution	sequential, possibly adversarial
Goal	low true error with high probability	few mistakes or sublinear regret
Feedback	training labels before final hypothesis	label/loss after each prediction
Output	final predictor	sequence of predictions

Relations:

- A finite mistake-bound online learner implies PAC learnability in the realizable case: simulate online learning on i.i.d. samples and use the final hypothesis.
- PAC learnability does not automatically imply low-mistake online learnability against adversarial sequences.
- Finite VC dimension characterizes distribution-free PAC learnability for binary classification.
- Finite Littlestone dimension characterizes online learnability in the realizable adversarial setting.

VC dimension

Set $S = \{x_1, \dots, x_m\}$ is **shattered** by H if for every labeling of S , there exists $h \in H$ realizing it.

The VC dimension $VC(H)$ is the size of the largest shattered set, or ∞ if arbitrarily large sets can be shattered.

For finite VC dimension d , a typical realizable PAC sample bound is

$$m = O\left(\frac{d \log(1/\epsilon) + \log(1/\delta)}{\epsilon}\right).$$

For agnostic learning, a typical bound is

$$m = O\left(\frac{d + \log(1/\delta)}{\epsilon^2}\right).$$

Necessary and sufficient condition: a binary class is distribution-free PAC-learnable iff it has finite VC dimension.

Simple explanation

VC dimension measures how many points a model class can label in every possible way. If the class can shatter too many points, it can memorize noise. Finite VC dimension means enough random examples will eventually control overfitting.

2. Learnability of Conjunctions and Disjunctions

Let instances be Boolean vectors $x \in \{0, 1\}^n$. Literals are variables x_i and negations $\neg x_i$.

Conjunctions in the PAC model

Class: conjunctions of literals, e.g.

$$c(x) = x_1 \wedge \neg x_3 \wedge x_5.$$

Efficient PAC algorithm for realizable data:

1. Start with all $2n$ literals in the hypothesis: $h = x_1 \wedge \neg x_1 \wedge \dots \wedge x_n \wedge \neg x_n$.
2. For every positive example x : remove every literal falsified by x .
3. Ignore negative examples.
4. Output remaining conjunction.

Reason: every literal falsified by a positive example cannot be in the target. Every remaining literal is consistent with all positive examples. If the sample is large enough, any harmful extra literal would probably be exposed by a positive example.

Efficient: $O(mn)$ time.

Example

With variables x_1, x_2 , start with $x_1 \wedge \neg x_1 \wedge x_2 \wedge \neg x_2$. Positive example $(x_1 = 1, x_2 = 0)$ removes $\neg x_1$ and x_2 . Positive example $(1, 1)$ removes $\neg x_2$. The remaining hypothesis is x_1 .

Disjunctions in the PAC model

Disjunctions can be learned directly by the dual algorithm:

1. Start with all literals in a disjunction.
2. For every negative example, remove literals made true by that example.
3. Ignore positive examples.

Or reduce disjunction learning to conjunction learning using De Morgan:

$$l_1 \vee \dots \vee l_k = \neg(\neg l_1 \wedge \dots \wedge \neg l_k).$$

Online learning of conjunctions

Use the same elimination idea:

1. Start with all literals.
2. Predict using current conjunction.
3. If a positive example is misclassified as negative, remove all literals falsified by it.
4. Negative mistakes do not require changing the hypothesis in the realizable monotone elimination version, but in the full literal class consistency must be maintained.

Each literal is removed at most once, so the number of updates is at most $2n$. Thus conjunctions are efficiently online learnable with a finite mistake bound.

Simple explanation

The conjunction learner starts overly strict and relaxes itself when positive examples prove that some literal cannot be required. Every mistake teaches it to delete at least one wrong requirement, so it cannot keep making the same kind of mistake forever.

Online learning of disjunctions

Use the dual algorithm or De Morgan reduction. Again the mistake/update bound is $O(n)$ in the realizable setting.

Learning by reduction

Many simple Boolean classes reduce to conjunctions/disjunctions:

- Monotone conjunctions: use only positive literals.
- Monotone disjunctions: dual.
- k -CNF: conjunction of clauses; treat each possible clause of size at most k as a Boolean feature, then learn a conjunction. Efficient for fixed k because the number of clauses is $O(n^k)$.
- k -DNF: reduce dually to k -CNF by negation.

The key trick is to transform the representation into a feature space where the target becomes a conjunction or disjunction.

3. Multi-Armed Bandits

Problem definition

There are K arms. At round $t = 1, \dots, T$:

1. learner chooses arm $A_t \in \{1, \dots, K\}$,
2. receives reward R_t drawn from the selected arm distribution,
3. observes only that reward.

Let arm means be $\mu_i = \mathbb{E}[R|A = i]$ and $\mu^* = \max_i \mu_i$.

Regret

Expected regret:

$$\mathbb{E}[R_T] = T\mu^* - \mathbb{E} \left[\sum_{t=1}^T R_t \right].$$

Equivalently:

$$\mathbb{E}[R_T] = \sum_{i: \Delta_i > 0} \Delta_i \mathbb{E}[N_i(T)],$$

where $\Delta_i = \mu^* - \mu_i$ and $N_i(T)$ is the number of times arm i was pulled.

Goal: balance **exploration** of uncertain arms and **exploitation** of arms currently believed best.

Epsilon-greedy

At each round:

- with probability $1 - \epsilon$, choose arm with largest empirical mean,
- with probability ϵ , choose a random arm.

With fixed ϵ , regret is linear because exploration never vanishes. With decaying ϵ_t , regret can improve, but the schedule is problem-sensitive.

Limitations:

- explores uniformly, wasting pulls on obviously bad arms,
- sensitive to ϵ ,
- weaker theoretical guarantees than confidence-bound or posterior-sampling methods.

UCB

Upper Confidence Bound chooses the arm with largest optimistic estimate:

$$A_t = \arg \max_i \left(\hat{\mu}_i(t) + \sqrt{\frac{2 \ln t}{N_i(t)}} \right).$$

The second term is an uncertainty bonus. Arms with few samples get high bonus; arms with many samples rely mostly on empirical mean.

For bounded rewards, UCB1 has logarithmic regret:

$$\mathbb{E}[R_T] = O\left(\sum_{i:\Delta_i>0} \frac{\ln T}{\Delta_i}\right).$$

Simple explanation

UCB is “optimism under uncertainty.” If an arm has not been tried much, pretend it might be better than it currently looks. As evidence accumulates, the uncertainty bonus shrinks and the algorithm naturally focuses on the best arms.

Example

At time $t = 100$, suppose arm A has empirical mean 0.6 after 25 pulls, and arm B has empirical mean 0.55 after 4 pulls. UCB gives B a larger uncertainty bonus because it has been tried less. The algorithm may choose B even though its current average is lower.

Thompson sampling

Thompson sampling is Bayesian probability matching:

1. Maintain posterior distribution over each arm’s mean.
2. Sample θ_i from each posterior.
3. Pull arm $\arg \max_i \theta_i$.
4. Update posterior using observed reward.

Example for Bernoulli arms:

- prior $\theta_i \sim \text{Beta}(\alpha_i, \beta_i)$,
- after success: $\alpha_i \leftarrow \alpha_i + 1$,
- after failure: $\beta_i \leftarrow \beta_i + 1$.

Advantages:

- simple,
- naturally balances exploration/exploitation,
- strong empirical performance and logarithmic regret under standard assumptions.

4. Reinforcement Learning

Markov Decision Process

An MDP is

$$(S, A, P, R, \gamma),$$

where S are states, A actions, $P(s'|s, a)$ transition probabilities, $R(s, a, s')$ rewards, and $\gamma \in [0, 1)$ discount factor.

A policy $\pi(a|s)$ defines behavior. The state-value function is

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \mid S_0 = s \right].$$

The action-value function is

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \mid S_0 = s, A_0 = a \right].$$

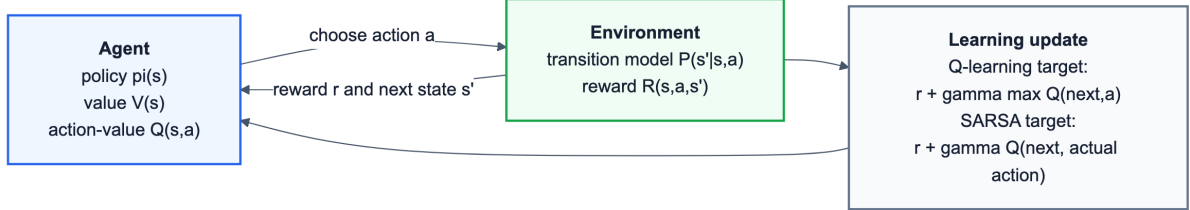


Figure 2: Reinforcement-learning loop: the agent chooses actions, the environment returns rewards and next states, and the value estimate is updated.

Optimal policy and Bellman equations

Optimal value:

$$V^*(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')].$$

An optimal policy chooses

$$\pi^*(s) \in \arg \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')].$$

Value iteration

For known model P, R :

$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_k(s')].$$

Repeat until values change little. Then extract greedy policy. Converges for discounted finite MDPs.

Direct utility estimation

Estimate $V^\pi(s)$ from sampled returns:

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots$$

Average observed returns following visits to state s . This is Monte Carlo prediction: unbiased but may have high variance and requires episodes or truncation.

Adaptive dynamic programming

Learn a model $\hat{P}, \hat{R}$ from experience, then solve the estimated MDP by dynamic programming. It is model-based RL:

1. count transitions and rewards,
2. estimate $\hat{P}, \hat{R}$,
3. run value iteration or policy iteration on the learned model.

Temporal difference learning

TD learns from bootstrapped one-step targets:

$$V(s) \leftarrow V(s) + \alpha (r + \gamma V(s') - V(s)).$$

TD combines Monte Carlo sampling with dynamic programming bootstrapping.

Exploration vs. exploitation

The agent must try actions to learn their consequences, but also use current knowledge to obtain reward.

Common strategies:

- ϵ -greedy,
- softmax/Boltzmann action selection,
- optimistic initialization,
- UCB-style exploration bonus,
- posterior sampling.

Q-learning

Off-policy TD control:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right).$$

It learns the greedy optimal value function independent of the behavior policy, assuming sufficient exploration and suitable learning rates.

Example

Let $Q(s, a) = 5$, reward $r = 2$, $\gamma = 0.9$, best next value $\max_{a'} Q(s', a') = 10$, and $\alpha = 0.1$. Target is $2 + 0.9 \cdot 10 = 11$. Update is $5 + 0.1(11 - 5) = 5.6$.

SARSA

On-policy TD control:

$$Q(s, a) \leftarrow Q(s, a) + \alpha (r + \gamma Q(s', a') - Q(s, a)),$$

where a' is the next action actually selected by the current behavior policy.

Difference from Q-learning:

- Q-learning target uses best possible next action.
- SARSA target uses actually chosen next action, so it accounts for exploratory behavior.

Exam tip

Q-learning is off-policy: it learns the greedy optimal policy even while behaving exploratorily. SARSA is on-policy: it learns the value of the policy it is actually following, including its exploratory moves.

Policy search

Instead of learning values, directly optimize parameterized policy π_θ :

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_t \gamma^t R_{t+1} \right].$$

Methods:

- finite-difference/random search,
- policy gradient,
- REINFORCE,
- actor-critic.

Policy search is useful for continuous actions and when value maximization is hard, but can have high variance and local optima.

Exam Checklist

- Define PAC learning and online learning and compare assumptions.
- State VC dimension and why it matters.
- Learn conjunctions/disjunctions by elimination and by reductions.
- Define bandit regret and explain epsilon-greedy, UCB, Thompson sampling.
- Define MDP, value functions, optimal policy, Bellman equation.
- Explain value iteration, Monte Carlo/direct utility estimation, ADP, TD.
- Compare Q-learning and SARSA.
- Explain exploration/exploitation and policy search.